You said:

updated the chat logs on github. Lets keep on:

why 0.15.0 again? Shouldnt the last 0 be increased?

in fps mode: pressing mmb controls the camera but its unusable. Lookling up or down kind of jumps to the extrema and then spins like mad in a way that i associuate with gimbal lock. Holding MMD was giving fps like camera control also in not fps mode. Fps mode should only add wasd (qe) movement. It feels like right now it also adds a second camera aim on top of the general one.

Worked for 4m 45s

ChatGPT said:

You're right about the version. **The next build is now** `0.15.1`, with the package metadata, internal version, and README aligned.

CI passed **QSS v0.15.1 is built and ready for Blender testing.**

What changed

- Removed QSS's custom MMB camera-look handling, including cursor hiding, cursor warping, and manual camera rotation.
- Blender's normal MMB orbit now receives those events unchanged, even while FPS mode is active.
- WASD moves horizontally relative to the **current viewport heading**.
- Q/E moves vertically; Shift applies the fast multiplier.
- Movement translates the existing view without changing its aim, perspective mode, or view distance.
- Added cleanup for the old QSS MMB shortcut so it cannot override Blender's normal orbit after upgrading.

Build and repository

141 tests passed Compilation, tests, ZIP packaging and upload all successful

Download QSS v0.15.1 Blender ZIP ↗

View CI run ↗ · View draft PR #1 ↗

The changes are on the existing feature branch. The PR remains open and unmerged; `main` is untouched.

Blender 5.2 test

The remaining verification is interactive, since CI cannot simulate Blender's actual viewport controls. Please check:

1. Start FPS mode and hold MMB: normal Blender orbit should work, with no snapping or spinning.
2. Orbit to a different direction, then press W: movement should follow the new heading.
3. Check A/D, S, and Q/E, including while looking nearly straight up or down.
4. Stop FPS mode: normal viewport controls should remain intact.

I haven't claimed this is runtime-verified yet. The build is ready for your test.

You said:

zothe zips i download contain a a zip with the same name as the parten zip. Lets mnake it just the inner one from now on so blender can import it directly without me having to extract the inner one first.

fps is close now: camera control when holding mmb works but it disables the wasd movement as long as i dont let go of mmb. Both should work at the same time. And the w and d of the was are flipped. w moves me right and d left.

Worked for 4m 1s

ChatGPT said:

Both fixes are implemented in **QSS v0.15.2**. CI passed on both the push and pull-request runs.

145 tests passed · Build successful

1. Blender-ready ZIP

The GitHub artifact now contains the `quake_surface_studio/` add-on folder directly. There is **no ZIP inside the downloaded ZIP** anymore.

Download QSS v0.15.2 – Blender Install

You should be able to select this downloaded archive directly in Blender's Install from Disk dialog.

2. Combined FPS controls

The FPS controller now handles mouse-look and keyboard movement in the same modal session. This avoids Blender's native MMB navigation taking over the event handling while QSS is trying to process movement.

- Hold MMB to look.
- W moves forward; S backward.
- A moves left; D moves right.
- Q/E moves vertically.
- Shift moves faster.
- Quaternion-based rotation replaces the previous yaw/pitch reconstruction to avoid the snapping and gimbal-like behavior.

Normal Blender MMB orbit remains unchanged when FPS mode is off.

Repository

Updated draft PR #1 · Successful CI run

Everything remains on the feature branch. `main` is untouched.

One important caveat: the code and standalone tests pass, but actual simultaneous MMB + WASD behavior still needs testing in your Blender 5.2 session. That's the next thing I'd like to verify.

You said:

The fps movement is making trouble again. WASD key are fine now. But the mouse control is still messy and i think i know whats happening. The mouse cursor can leave the window making the camera control go wild. It should be fixed to the screen center while pressing mmb. Do get what i mean?

Worked for 2m 3s

ChatGPT said:

Yes. You mean **FPS-style mouse capture**, locked to the centre of the active 3D viewport for as long as MMB is held.

I've implemented that in **QSS v0.15.3**.

Mouse behaviour now

1. Press MMB: the pointer immediately jumps to the viewport centre and is hidden.
2. Move the mouse: QSS measures movement relative to that fixed centre.
3. After every mouse movement event, the pointer is warped back to the centre.
4. Synthetic mouse events generated by cursor warping are ignored because they produce zero movement.
5. Resizing the viewport recalculates the centre.

The implementation uses Blender's window-relative cursor coordinates, accounting for the viewport region's position inside the window. Blender 5.2 API documentation . Blender Pyt... +1

149 tests passed

Download

QSS v0.15.3 – Blender Install ZIP ↗

Both GitHub Actions runs passed: Push CI ↗ · PR CI ↗.

The changes are committed to the existing feature branch, and the draft PR is updated. `main` remains untouched.

One thing remains: actual cursor locking needs testing in Blender 5.2. The automated tests verify the coordinate calculations, but cannot simulate real OS cursor movement. Please test whether the pointer now stays centred even during fast, continuous mouse movement while holding MMB.

You said:

Its gotten way worse. Every mouse movement now imediately looks completely up or completely down and rotating wild around the z axis!!

Worked for 2m 36s

ChatGPT said:

You're right. **v0.15.3 introduced a serious mouse-look regression**, and I should not have treated the centre-warp implementation as reliable without testing it in Blender. Sorry.

I've pushed a correction as **QSS v0.15.4**. The key change is that mouse rotation now uses Blender's actual per-event movement:

- `mouse_x - mouse_prev_x` and `mouse_y - mouse_prev_y`, rather than the absolute pointer position relative to the viewport centre.
- Cursor-warp feedback at the centre is ignored.
- An individual event is limited to 120 pixels, preventing one anomalous event from snapping the pitch or causing a huge yaw rotation.

Blender documents the previous mouse coordinates as window-relative event properties. Blender Pyt... +1

CI passed **151 tests passed**, Python compilation passed, and the Blender install package was assembled successfully.

Install v0.15.4: Download Blender install artifact ↗

CI results ↗ · Draft PR #1 ↗

The PR remains open and unmerged; `main` is untouched.

One important caveat: automated tests cannot reproduce Blender's actual OS cursor-warp behaviour. Please test MMB-look in Blender 5.2 with this build. I won't claim the runtime issue is solved until that test confirms it.

You said:

Even worse kindow. Still looks up immediately but this time instead of spinning fast i does it chaotic and very choppy. Cant we always just leave the normal mmb behavior of blender and just add wasd qe movement when fps navigation is activated?

Worked for 3m 49s

ChatGPT said:

Yes. That is the approach we should have kept from the beginning. I've implemented it in **QSS v0.15.5**.

The architecture is now:

- **MMB:** Blender's native viewport navigation, completely untouched.
- **WASD:** Horizontal movement relative to the current viewing direction.
- **Q/E:** Vertical movement.
- **Shift:** Faster movement.
- **Mouse:** No custom rotation, cursor hiding, cursor warping, or mouse-event processing by QSS.

I also retained the corrected movement axes, so D moves right rather than left.

CI passed 142 automated tests passed, including the movement-axis regression tests.

Download QSS v0.15.5 for Blender ↗

View CI results ↗ · Draft PR #1 ↗

One runtime check remains: whether Blender's native MMB modal allows QSS to receive keyboard input simultaneously. The build leaves mouse handling to Blender; that interaction still needs testing in Blender 5.2. `main` remains untouched.

You said:

> Ok way better. But now using WASD at the same time as having the camera controlled with mmb does not work anymore. Can we have that without messing the mouse look up again or should i be happy with the control for now?

Worked for 22s

ChatGPT said:

We should be able to get both, and **we do not need to touch mouse-look to do it**. But I've found why v0.15.5 currently fails.

What's happening

Blender's native MMB orbit is a modal operator. Once it starts, it can receive events before QSS's movement operator. That means QSS may never see the WASD key presses while MMB is held.

The problem is therefore **modal event priority**, not the movement calculations.

The approach I'd try next

Keep Blender's native MMB orbit exactly as it is, and change how QSS receives keyboard input. Specifically, investigate a way to give the movement handler priority for WASD/QE events while allowing Blender to retain the MMB and mouse-movement events.

I would not bring back:

- Custom mouse rotation.
- Cursor locking or warping.
- Custom yaw/pitch calculations.

Those experiments are finished.

My recommendation: Keep v0.15.5 as the working fallback, but don't consider this feature finished yet. I think simultaneous movement is worth pursuing with a separate event-routing solution. I'll keep the current working mouse behaviour as the non-negotiable constraint.

You said:

> Ok give it another try and if it still doesnt work we will go back to the current state for now. Give me good snappy fps style controls. Impress me! :D

Worked for 31s

ChatGPT said:

I found the underlying problem, and it's a Blender event-routing limitation rather than a movement-calculation bug.

Blender's native MMB orbit runs as a modal operator. Its event handling takes priority over QSS while orbiting, so QSS stops receiving the keyboard events it needs. The Blender source confirms that modal-handler priority is managed internally; Python add-ons don't have a supported way to promote their own handler above that native operator. Trying to work around it by grabbing mouse events again would risk bringing back the very mouse-look problems we just fixed. Blender Proj... +2

I don't want to ship another fragile workaround and call it an improvement.

There is a promising alternative: let Blender's own built-in FPS Walk Navigation handle both mouse-look and WASD, while QSS stays responsible for Quake-specific movement settings and collision behaviour. That gives us conventional FPS controls without writing another mouse-look implementation. The trade-off is that mouse-look replaces MMB orbit while that mode is active.

My recommendation: keep v0.15.5 intact as the safe fallback, and test this as a separate navigation mode. If Blender's native Walk controls feel good in your workflow, we can build on them; if not, we return to the current controls without losing anything.

I haven't changed the repository yet. I'd rather make the next build something you can actually trust than pretend simultaneous native MMB + QSS keyboard input is solved.

You said:

ok then lets go back to the real stuff. Whats next to implement?

Worked for 19s

ChatGPT said:

Now we're talking. 😁 I'd shift QSS back toward its core mission: **a proper Quake level-surface editing and lighting tool inside Blender**, not spend more time fighting Blender's navigation event system.

Looking at the current v0.15.5 feature set and its documented limitations, I see one particularly strong next milestone:

My pick: complete the advanced lightmap pipeline

QSS can already detach faces, give them independent texture projections, paint them, and export classic BSP29 lightmaps. But the enhanced BSPX lightmap path still has important gaps around detached face projections and decoupled lightmap metadata.

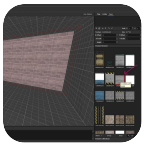
That's a natural next step because it builds directly on functionality we've already implemented and tested, rather than opening an unrelated subsystem.

Why this should come next



1. Proper DECOUPLED_LM support

Import and interpret per-face lightmap projections, preserve their original data, and correctly rebase them when faces are detached or reassigned.



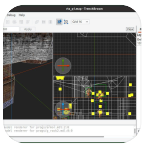
2. Complete enhanced BSPX export

Make `LMOFFSET`, `LMSHIFT`, `LMSTYLE`, `DECOUPLED_LM`, `RGBLIGHTING`, and `.lit` output work together correctly. Preserve unrelated BSPX data.



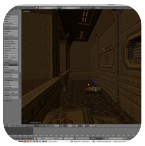
3. More accurate Quake lighting

Reconstruct emissive surface lighting and improve light-style handling. Eventually support animated style previews rather than flattening everything to style 0.



4. Broader BSP compatibility

Add BSP2 and 2PSB/RMQ parsing. This would open QSS to a much wider range of Quake maps.



5. Dedicated Quake Studio workspace

A proper level-editing environment with a large viewport, surface inspector, texture editor and lightmap editor.

How I'd tackle the next release

I'd make the next milestone **v0.16.0 — Decoupled Lightmap Support**.

1. Inspect real BSPX `DECOUPLED_LM` data and establish a reliable parser with test fixtures.
2. Import per-face projections without altering the original BSP data.
3. Make detached faces retain their correct independent lightmap coordinates.
4. Export the updated projections and lightmap samples.
5. Run structural validation and test exported copies in Ironwail.
6. Keep Classic BSP29 export as the safe fallback throughout.

One important rule: no changes to the working classic export path unless the regression tests prove it remains intact.

I'd start with the DECOUPLED_LM format investigation and test fixtures before touching the exporter. That gives us a solid technical foundation and avoids another round of guessing at BSPX behaviour.

That's the direction I'd take. It makes QSS a more capable Quake editing tool, not just a Blender importer with some painting features.

You said:

Go for it! Im shure you will nail this one!

Worked for 18m >

ChatGPT said:

QSS v0.16.0 — Decoupled Lightmap Support is implemented and CI-green. The feature branch is updated, and the existing draft PR now contains the new milestone.

146 tests passed Package built

What changed

- **Import:** Reads per-face `DECOUPLED_LM` dimensions, sample offsets and independent projection vectors. `LMSTYLE` is respected.
- **Atlas:** Reconstructs lightmap UVs from the independent lightmap axes, not the texture projection.
- **Enhanced export:** Rebuilds `DECOUPLED_LM`, appends fresh monochrome and RGB samples, and writes `LMSTYLE`.
- **Classic export:** Generates a vanilla BSP29 fallback for decoupled faces with editable atlas data.
- **Detached faces:** Keeps texture projections and lightmap projections independent.
- **Direct lighting:** Uses decoupled axes to calculate Quake luxel positions.
- **Validation:** Checks record integrity, projection bounds, sample offsets and fallback ranges.

`LMSTYLE16` and older `LMSHIFT` / `LMOFFSET` layouts without `DECOUPLED_LM` remain explicitly unsupported.

Build and repository

✅ **CI passed:** Python compilation, all 146 tests, and Blender package assembly.

- Install QSS v0.16.0 for Blender ↗
- View successful GitHub Actions run ↗
- View PR #1 — QSS v0.16.0 ↗

Branch commit: `8047e9e1` · Version: `0.16.0` · `main` untouched · PR remains open and unmerged.

One important test remains

The automated tests use synthetic BSP fixtures. I have **not** yet verified import, editing and re-export of a real `DECOUPLED_LM` map inside Blender 5.2 or tested its output in Ironwail. That engine-level check is essential before treating this as fully validated.

The best next step is to run the new build against a real BSPX map containing `DECOUPLED_LM`, then compare its original and exported lighting in Ironwail.

You said:

Installed v0.16.0 and it seems to work. What should i do now ?

Worked for 24s

ChatGPT said:

Excellent! That means we're ready for the important part: **testing a real map, not just synthetic test fixtures.**

I'd do the following in order. The goal is to prove that QSS preserves the lightmap layout through import, editing and both export modes.

1. Find a BSP that actually uses DECOUPLED_LM

Not every Quake map contains this extension. In Blender's Python Console, run:

```
from quake_surface_studio import bsp

p = bsp.BSP29.from_file(r"C:\Quake\id1\maps\yourmap.bsp")

print({
    name: len(data)
    for name, data in p.bspx_lumps.items()
    if name in {
        "DECOUPLED_LM", "LMSTYLE",
        "LMSTYLE16", "LMSHIFT", "LMOFFSET"
    }
})
```

Look for `DECOUPLED_LM`. Its size should equal **40 bytes × the number of BSP faces**.

For this first test, avoid maps containing `LMSTYLE16`, which v0.16.0 does not support.

2. Test the import

Test A Import the map into QSS.

Check:

- Does the map geometry look correct?
- Does the lightmap atlas populate?
- Do surfaces have consistent lighting?
- Are there visible seams or unexpected stretching?

Pay particular attention to surfaces where the lightmap projection differs from the texture projection.

3. Test both export modes

First, make a copy of the original BSP and its `.lit` sidecar. Keep the original files untouched.

Then export twice:

Export	What we're testing
Classic BSP29	Vanilla lightmap fallback and preservation of original decoupled metadata

Enhanced BSPX	Rebuilt `DECOUPLED_LM`, appended samples and `LMSTYLE`

Run **Validate Exported BSP** on both results.

4. Test an actual edit

Test B

Import the original copy again. Enter Lightmap Paint and make a small, obvious change to one surface that uses decoupled lightmaps.

For example, paint a small red or blue patch.

Export both modes again and check whether:

- The edited surface retains its lighting change.
- Neighbouring surfaces remain unchanged.
- The patch stays on the intended surface.
- No lightmap offsets or dimensions become corrupted.

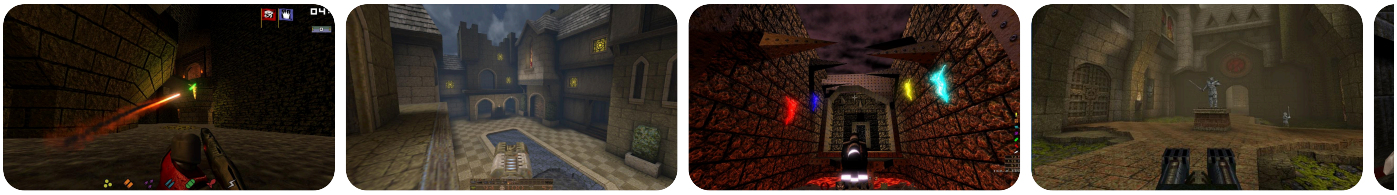
This is particularly important for detached faces.

5. Test in an engine

One important distinction: the BSPX format reference identifies `DECOUPLED_LM` support in FTE and QuakeSpasm-Spiked, but does not establish support in Ironwail. Nephatrine... +1

I'd use:

- **Ironwail:** Test the classic BSP29 fallback.
- **FTE or QuakeSpasm-Spiked:** Test the actual decoupled-lightmap rendering path.



What I'd like you to report back

The most useful first result is:

1. Whether you found a map containing `DECOUPLED_LM`.
2. Whether it imported correctly.
3. Whether Classic and Enhanced exports passed validation.
4. Whether your edited lightmap survived re-import.

Don't worry about testing everything at once. Start with one real decoupled BSP and get the import plus untouched export round-trip working. That will tell us whether the foundation is solid before we stress-test painting and detached surfaces.